

CSE 4403: Algorithms

Lecture13

Dynamic Programming - II

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

What even is dynamic programming?

Dynamic Programming (DP) sounds intimidating. It's not. At its heart, it's one simple idea:

CORE IDEA

Don't solve the same sub-problem twice. Break a big problem into smaller pieces, solve each piece once, remember the answer, and reuse it.

That's it. Everything else — tables, recurrences, optimal substructure — are just details of *how* to do this efficiently.

Two Necessary Ingredients

INGREDIENT 1

Overlapping Subproblems

The same smaller problem appears many times. Without this, DP gives no benefit over plain recursion.

INGREDIENT 2

Optimal Substructure

The best solution to a big problem is built from best solutions of smaller problems.

Tabulation: Fibonacci Sequence

(Bottom-Up DP)

```
dp[0]=0;  
dp[1]=1;  
  
for(int i=2;i<=n;i++)  
    dp[i]=dp[i-1]+dp[i-2];
```

The DP Recipe

How to solve a DP problem

- Define the state
- Define the recurrence
 - Means: How to combine the previous results to get current?
- Identify base cases
- Compute/store answers
- Return the desired state

Example (Fibonacci):

- State: $dp[i]$ = i th Fibonacci number
- Recurrence: $dp[i] = dp[i-1] + dp[i-2]$
- Base: $dp[0] = 0$, $dp[1] = 1$
- Answer: $dp[n]$

Home Task Solution

Problem:

- You have coins: 1, 3, 4
- Find the minimum number of coins needed for spending **6 units**.
- **A coin denomination may be used any number of times.**

Home Task

Problem:

- You have coins: 1, 3, 4
- Find the minimum number of coins needed for spending **6 units**.

A coin denomination may be used any number of times.

Solve using DP:

Find the state.

Find the recurrence relation.

Find the base cases.

...

Home Task Solution

Step 1: Define the state

Let

$dp[x]$ = minimum number of coins needed to make value x

We want:

$dp[6]$

Step 2: Recurrence

For every value x , try taking one coin:

$$dp[x] = \min(\begin{aligned} &dp[x-1], \\ &dp[x-3], \\ &dp[x-4], \\ &) + 1 \end{aligned}$$

(only if the indices are non-negative)

Home Task Solution

Step 1: Define the state

Let

$dp[x]$ = minimum number of coins needed to make value x

We want:

$dp[6]$

Step 2: Recurrence

For every value x , try taking one coin:

$$dp[x] = \min(\begin{aligned} &dp[x-1] + 1, \\ &dp[x-3] + 1, \\ &dp[x-4] + 1 \end{aligned})$$

(only if the indices are non-negative)

Home Task Solution

Step 3: Base case

To make value 0:

$$dp[0] = 0$$

No coins are needed.

x=1:

$$dp[1] = ?$$

$$\begin{aligned} \text{coin 1: } dp[1 - 1] + 1 \\ &= dp[0] + 1 \\ &= 0 + 1 \end{aligned}$$

We cant use coin 3 or 4.

x=2:

$$dp[2] = ?$$

$$\begin{aligned} \text{coin 1: } dp[2 - 1] + 1 \\ &= dp[1] + 1 \\ &= 2 \end{aligned}$$

x=3:

$$\text{coin 1: } dp[3-1] + 1 = 3$$

$$\text{coin 3: } dp[3-3] + 1 = 1$$

$$dp[3] = \min(3, 1) = 1$$

x=4:

Choices:

$$\text{coin 1: } dp[4-1] + 1 = dp[3] + 1 = 2$$

$$\text{coin 3: } dp[4-3] + 1 = dp[1] + 1 = 2$$

$$\text{coin 4: } dp[4-4] + 1 = dp[0] + 1 = 1$$

$$dp[4] = \min(2, 2, 1) = 1$$

Home Task Solution

Step 4: Fill the table

Until desired state reached

Step 5: Return the desired state

Home Task Solution: Recursive

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int solve(int x) {
    if (x == 0) return 0;
    if (x < 0) return INF;

    return 1 + min({
        solve(x - 1),
        solve(x - 3),
        solve(x - 4)
    });
}

int main() {
    cout << solve(6) << '\n';
}
```

Home Task Solution: Memoization

```
int solve(int x) {  
    if (x == 0) return 0;  
    if (x < 0) return INF;  
  
    return 1 + min({  
        solve(x - 1),  
        solve(x - 3),  
        solve(x - 4)  
    });  
}
```

```
int solve(int x) {  
    if (x == 0) return 0;  
    if (x < 0) return INF;  
  
    if (dp[x] != -1)  
        return dp[x];  
  
    return dp[x] = 1 + min({  
        solve(x - 1),  
        solve(x - 3),  
        solve(x - 4)  
    });  
}
```

Home Task Solution: Tabulation

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int target = 6;

    vector<int> dp(target + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= target; i++) {
        if (i >= 1)
            dp[i] = min(dp[i], dp[i - 1] + 1);

        if (i >= 3)
            dp[i] = min(dp[i], dp[i - 3] + 1);

        if (i >= 4)
            dp[i] = min(dp[i], dp[i - 4] + 1);
    }

    cout << dp[target] << '\n';
}
```

Home Task Solution: Tabulation(coins are given as input)

```
int main() {  
    vector<int> coins = {1, 3, 4};  
    int target = 6;  
  
    vector<int> dp(target + 1, INT_MAX);  
  
    dp[0] = 0;  
  
    for (int i = 1; i <= target; i++) {  
        for (int coin : coins) {  
            if (i >= coin && dp[i - coin] != INT_MAX) {  
                dp[i] = min(dp[i], dp[i - coin] + 1);  
            }  
        }  
    }  
  
    cout << dp[target] << '\n';  
  
    return 0;  
}
```

Home Task Solution

Amount	0	1	2	3	4	5	6
Min Coins	0	1	2	1	1	2	2

Number of Permutations

Problem:

- You have coins: 1, 3, 4
- Find the number of ways to make 6.
- **A coin denomination may be used any number of times.**

- **State**

$dp[x]$ = number of ways to make amount x

- **Base Case**

There's exactly one way to make amount 0:

$dp[0] = 1$

Number of Permutations

Problem reformulated:

- Given coins of different denominations and a target amount, count the number of **distinct permutations** that sum to the target. **For now, lets say order matters — {1,2} and {2,1} are not the same.**

- **State**

$dp[x]$ = number of ways to make amount x

- **Base Case**

There's exactly one way to make amount 0:

$dp[0] = 1$

Number of Permutations

- **Recurrence**

For each coin:

$dp[x] += dp[x - \text{coin}]$

```
int main() {  
    int n = 6;  
    vector<long long> dp(n + 1, 0);  
  
    dp[0] = 1;  
  
    for (int i = 1; i <= n; i++) {  
  
        long long ways = 0;  
  
        if (i - 1 >= 0) ways += dp[i - 1];  
        if (i - 3 >= 0) ways += dp[i - 3];  
        if (i - 4 >= 0) ways += dp[i - 4];  
  
        dp[i] = ways;  
    }  
  
    cout << dp[n] << '\n';  
}
```

Coin Change: Number of Permutations

- **Recurrence**

For each coin:

$$dp[x] += dp[x - \text{coin}]$$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> coins = {1, 3, 4};
    int n = 6;

    vector<int> dp(n + 1, 0);
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        for (int c : coins) {
            if (i >= c)
                dp[i] += dp[i - c];
        }
    }

    cout << dp[n] << '\n';
}
```

Coin Change/Number of Combinations (Order of coins doesn't matter)

- **Recurrence**

For each coin:

$dp[x] += dp[x - \text{coin}]$

```
int main() {  
    int target = 6;  
  
    vector<long long> dp(target + 1, 0);  
    dp[0] = 1;  
  
    // coin = 1  
    for (int i = 1; i <= target; i++)  
        dp[i] += dp[i - 1];  
  
    // coin = 3  
    for (int i = 3; i <= target; i++)  
        dp[i] += dp[i - 3];  
  
    // coin = 4  
    for (int i = 4; i <= target; i++)  
        dp[i] += dp[i - 4];  
  
    cout << dp[target] << '\n';  
}
```

Coin Change/Number of Combinations (Order of coins doesn't matter)

Caution:

The order of the loop has to be reversed.

Why? ---- After mid

```
int main() {  
    vector<int> coins = {1, 3, 4};  
    int n = 6;  
  
    vector<long long> dp(n + 1, 0);  
    dp[0] = 1;  
  
    for (int coin : coins) {  
        for (int sum = coin; sum <= n; sum++) {  
            dp[sum] += dp[sum - coin];  
        }  
    }  
  
    cout << dp[n] << '\n';  
}
```

Friend's Pairing Problem:

- Given n friends, each one can remain single or can be paired up with some other friend. Each friend can be paired only once. Find out the total number of ways in which friends can remain single or can be paired up.

Friend's Pairing Problem:

- Total number of ways to make friends with n people:
 - N^{th} person may decide to remain single
 - Or
 - N^{th} person may decide to pair up with remaining $(n-1)$ person

Input : $n = 3$

Output : 4

Explanation: $\{1\}, \{2\}, \{3\}$: all single

$\{1\}, \{2, 3\}$: 2 and 3 paired but 1 is single.

$\{1, 2\}, \{3\}$: 1 and 2 are paired but 3 is single.

$\{1, 3\}, \{2\}$: 1 and 3 are paired but 2 is single.

Note that $\{1, 2\}$ and $\{2, 1\}$ are considered same.

Friend's Pairing Problem:

```
int countFriendsPairings(int n)
{
    int dp[n + 1];

    // Filling dp[] in bottom-up manner using
    // recursive formula explained above.
    for (int i = 0; i <= n; i++) {
        if (i <= 2)
            dp[i] = i;
        else
            dp[i] = dp[i - 1] + (i - 1) * dp[i - 2];
    }

    return dp[n];
}
```